CAREERTRACK

Full-Stack Job Portal & Recruitment Management System

MAJOR PROJECT REPORT

Student Name	Nikhat Siddekee
Registered Email	nikhatsiddekee3@gmail.com
Program	Full Stack Web Development / MERN Stack Training Program
Batch	September 2026
Roll Number	WD-FSWD-A4/SEP-12005
Project Type	Major Project

Project Repository

https://github.com/Nikhat-codes/CareerTrack

Live Application

https://career-track-two.vercel.app

Backend Health Check

https://careertrack-vvpy.onrender.com/api/health

Submitted as part of the September 2026 training batch

REPORT CONTENTS

- 1. Project Title and Student Details
- 2. Abstract
- 3. Introduction and Problem Statement
- 4. Project Objectives and Scope
- 5. Functional and Non-functional Requirements
- 6. Technology Stack
- 7. System Architecture and Data Flow
- 8. Database Schema and Relationships
- 9. API Design and Endpoint Overview
- 10. Frontend Implementation
- 11. Backend Implementation
- 12. Authentication, Authorization and Security
- 13. File Upload Handling and Validation
- 14. Testing Methodology and Test Results
- 15. Screenshots of Major Modules
- 16. Challenges Encountered and Solutions
- 17. Limitations and Future Enhancements
- 18. Conclusion
- 19. References and Resources

Note: This report documents the implementation details available in the project README, API handover notes, test evidence and development history. Where exact source-level details were not available for independent confirmation, the report describes the feature at a functional level rather than inventing implementation specifics.

1. Project Title and Student Details

Project title

CareerTrack – Full-Stack Job Portal & Recruitment Management System.

Student

Nikhat Siddekee.

Program and batch

Full Stack Web Development / MERN Stack Training Program, September 2026 batch.

Roll number

WD-FSWD-A4/SEP-12005.

Project summary

A web-based recruitment platform with a public job-seeker experience and a protected administrator workspace.

2. Abstract

CareerTrack is a MERN-stack recruitment portal designed to bring job discovery, application submission and recruitment administration into one web application. Visitors can browse published job openings, view job details, submit an application with a resume without creating a candidate account, and track application progress using an application ID and email address. Administrators can sign in to a protected dashboard, review applications, manage job postings, update application statuses, save internal notes, download resumes and export application data. The frontend is built with React and Vite. The backend uses Node.js and Express, with MongoDB accessed through Mongoose. JWT-based authentication and bcrypt password hashing are used for administrator access. Multer handles resume uploads. The project is deployed with the frontend on Vercel and the backend on Render.

3. Introduction and Problem Statement

Introduction

Recruitment workflows involve publishing vacancies, receiving candidate information, reviewing applications and communicating progress. When these activities are handled through separate channels or manually maintained records, it can become difficult to keep information organized and provide a consistent application experience.

Problem statement

A centralized system is needed to make job opportunities accessible, collect applications in a structured format, store supporting resumes, and give administrators a manageable way to review candidates and update progress. CareerTrack addresses this workflow through a public portal connected to an administrative recruitment interface.

Intended users

The system has two main user groups: public visitors/job applicants and authorized administrators. Candidate registration is not required for the application workflow described in the project documentation.

4. Project Objectives and Scope

Objectives

Provide a responsive public job-browsing experience; allow candidates to apply online and attach a resume; support application tracking using application ID and email; provide authenticated administrative access; support job-posting management; and provide application review, status updates, notes, resume download, CSV export and dashboard statistics.

Included scope

Public job listing, search and job details; application form and resume upload; application tracking; administrator login; dashboard statistics; job creation, editing and publication status management; application listing and filtering; status and notes updates; resume download; and CSV export.

Out of scope / not claimed

The available project notes do not establish automated candidate ranking, payment processing, employer self-registration, email/SMS notifications or an AI-based matching engine. These are not presented as implemented features.

5. Functional and Non-functional Requirements

Functional requirements

Public users can view published jobs, search or filter listings, open job details, submit applications with a resume and track an application. Administrators can authenticate, view dashboard statistics, manage job postings, review applications, filter records, update status and notes, download resumes and export application data.

Non-functional requirements

Usability: pages should present job and application information clearly. Responsiveness: the interface is designed for common desktop and mobile viewport sizes. Security: administrative operations are protected and credentials are not stored in the public repository. Maintainability: frontend, backend, routes, controllers, models and middleware are separated into project modules. Availability: the frontend and backend are deployed independently.

Operational constraints

The backend is hosted on Render and may take time to respond after inactivity. Resume storage is currently described as local disk storage, which needs persistent or cloud storage for production reliability.

6. Technology Stack

Frontend

React for the user interface, Vite for the frontend development/build toolchain, React Router for client-side navigation, and Axios for HTTP communication.

Backend

Node.js runtime and Express.js for REST API routing and server-side request handling.

Database

MongoDB for persistence and Mongoose for object modelling and database access.

Authentication and uploads

JSON Web Token (JWT) for administrator authentication, bcrypt for password hashing, and Multer for handling resume uploads.

Testing and deployment

Postman is used for API testing resources. The frontend is deployed on Vercel and the backend is deployed on Render.

7. System Architecture and Data Flow

Architecture overview

CareerTrack follows a client-server architecture. The React application runs in the browser and communicates with the Express API over HTTP. The backend applies route and middleware checks, executes controller logic and uses Mongoose to read or write MongoDB records. Uploaded resumes are handled by the backend upload middleware.

Public application flow

A visitor opens the job portal, browses a published vacancy and opens its details. The visitor completes the application form and attaches a resume. The frontend sends the application request to the backend. The backend validates and processes the upload, stores application information through the data layer and returns a response. The applicant can later use the tracking form with the application ID and email.

Administrator flow

An administrator submits login credentials. The backend verifies the credentials and issues a JWT on successful authentication. Subsequent protected requests include the bearer token. The backend verifies the token before allowing dashboard, job-management or application-management operations.

Data-flow diagram

Browser (React UI) → HTTP request (Axios) → Express API routes → Authentication/upload middleware (where required) → Controllers → Mongoose models → MongoDB. Responses travel back through Express to the React interface. Resume upload/download and CSV export are file-oriented flows.

8. Database Schema and Relationships

Database overview

The project documentation identifies MongoDB with Mongoose and three principal data areas: jobs, applications and administrator accounts.

Jobs collection

Represents published or managed vacancies. The application supports creating and editing job postings and changing publication status. Exact field names and schema types should be taken from the final Mongoose model files.

Applications collection

Represents candidate submissions and the recruitment status. Application records are associated with the relevant job and include candidate-submitted information and resume reference/metadata. Administrators can update status and internal notes.

Admin collection

Represents administrator accounts used for protected access. Passwords are hashed using bcrypt; the login process uses JWT authentication.

Relationships

A job can be associated with multiple applications. Each application belongs to a job. Administrator accounts access the management workflow; the available documentation does not establish a separate ownership relationship between an admin and each application.

Schema note

Exact field-level schema definitions, indexes, required flags and enum values must be read from the current Mongoose model files. This report avoids fabricating field names or database indexes that were not present in the supplied documentation.

9. API Design and Endpoint Overview

API conventions

The frontend communicates with the backend through REST-style HTTP endpoints. Ordinary requests and responses use JSON. Resume downloads and CSV export return file responses. Protected administrator endpoints require an Authorization: Bearer token.

Public endpoints

GET /api/jobs — retrieve public published-job listings and supported search/filter results. GET /api/jobs/:id — retrieve an individual job. POST /api/applications — submit a candidate application using multipart/form-data with the resume field. POST /api/applications/track — track an application using application ID and email.

Administrator endpoints

POST /api/auth/login — administrator login. GET /api/dashboard/stats — protected dashboard statistics. GET /api/applications — protected application list with supported filters/pagination. PATCH /api/applications/:id/status — update application status and internal notes. GET /api/applications/export/csv — export application data. GET /api/applications/:id/resume — download a resume. DELETE /api/applications/:id — delete an application.

Job administration

Job administration routes are present in Backend/src/routes/adminJobRoutes.js. The precise HTTP methods and paths are not repeated here because the available API handover notes specifically flag those route mounts for source-level confirmation.

Endpoint verification note

Before submission, compare the endpoint list with the current backend route-mount files and Postman collection. This is especially important for job-administration routes, request field names, response examples and pagination parameters.

10. Frontend Implementation

Frontend structure

The frontend repository uses React and Vite. The documented structure includes src/pages, src/services, App.jsx, App.css, index.css and main.jsx. React Router is used for navigation and Axios for API communication.

Public portal

The public-facing interface provides job discovery, job details, an application form and an application tracking flow. Candidates can submit an application without creating a separate account.

Administrative interface

The administrator-facing interface includes login, dashboard statistics, job management and application management. Application management supports filters and status/notes updates. The interface also provides resume download and CSV export actions.

Navigation and state

The frontend routes connect the public pages and protected administrative experience. The available project notes confirm a TrackApplication route at /track-application.

11. Backend Implementation

Server and API

The backend is implemented with Node.js and Express. It exposes REST API endpoints for public job discovery, application submission and tracking, and protected administrative operations.

Modular organization

The repository structure separates backend source into config, controllers, middleware, models, routes and utils. This separation helps keep database setup, request handling, validation and business logic organized.

Database access

Mongoose provides the data-model layer for MongoDB. The backend stores job, application and administrator information in the database.

Operational deployment

The backend is deployed on Render. The health endpoint is available at https://careertrack-vvpy.onrender.com/api/health and returns a success response indicating that the API is running.

12. Authentication, Authorization and Security

Authentication

Administrator login is handled through POST /api/auth/login. Passwords are hashed using bcrypt. On successful login, the backend issues a JWT that is used for subsequent protected requests.

Authorization

Protected dashboard and management operations require a bearer token in the Authorization header. The backend verifies the token before allowing access to protected resources.

Data protection

Production secrets and actual .env files must not be committed to GitHub. Candidate personal data and real resumes should not be used in public screenshots or repository samples. Administrative credentials should not be published in the report or public repository.

Security improvements

The project handover notes identify further work such as confirming CORS configuration and secrets, adding file content-signature validation, and defining data-retention and audit-logging policies. These are treated as improvement areas rather than completed features.

13. File Upload Handling and Validation

Upload mechanism

Multer is used to handle candidate resume uploads submitted with an application. The API handover notes describe PDF, DOC and DOCX as accepted document types and a 5 MB upload limit.

Application workflow

The resume is attached to the public application submission request. Administrators can retrieve a resume through the protected resume-download endpoint.

Validation and safety

The documented upload checks include file type and size. The project notes recommend adding content-signature validation in addition to extension/MIME checks. Resume access should remain protected and actual candidate documents must not be committed to source control.

Storage limitation

Resume storage is currently described as local disk storage. A persistent or cloud-based storage service is recommended for a production deployment where the hosting filesystem may be ephemeral.

14. Testing Methodology and Test Results

Testing approach

The project was checked through functional workflows in the browser and API checks during development. Postman resources and a Test_Cases.xlsx file are included in the documented repository structure.

Evidence available

The development handover records a successful frontend production build, a live backend deployment for application filters, and a live combined job/date/status filter returning one matching application. Previously verified workflows include application submission, administrator review, status persistence, resume download, application tracking and CSV export.

Test summary

The test summary below reflects only the verification evidence recorded in the project handover. It is not a claim that every possible negative, security or cross-device test has passed.

Recommended final checks

Before final handover, execute the complete test workbook against the current deployment. Include valid and invalid login, unauthorized access, job creation/editing/publication, application submission, missing/invalid/oversized resume, tracking with mismatched email, status/notes persistence after refresh, resume download authorization, CSV export and error handling.

15. Screenshots of Major Modules

Evidence note

The following screenshots are included as visual evidence of the project interface captured during development. They show the public portal, application-management screens and administrator dashboard/job-management views.

16. Challenges Encountered and Solutions

Coordinating public and admin workflows

The system has two distinct experiences. The solution is to keep public browsing/application routes separate from protected management routes and use backend authorization for administrative operations.

Application status visibility

A candidate needs a way to check progress without an account. The project provides application tracking using application ID and email.

Resume handling

Resume upload and download require multipart form handling and protected retrieval. Multer is used for upload processing; persistent storage and stronger content-signature checks remain improvement areas.

Deployment reliability

The frontend and backend are deployed separately. A backend health endpoint provides a basic availability check; Render free-service inactivity can cause a cold-start delay.

17. Limitations and Future Enhancements

Current limitations

Resume storage uses local disk and may not persist reliably across hosting restarts or redeployments. File content-signature validation is identified as a needed improvement. Comprehensive responsive, security and negative-path testing remains to be completed. Data-retention and audit-logging policies are not yet defined in the supplied handover notes.

Future enhancements

Move resumes to persistent/cloud object storage; add file signature and malware scanning; add audit logs for administrative actions; define data-retention rules; expand automated tests; improve accessibility and responsive coverage; and add notifications if required by future project scope.

18. Conclusion

CareerTrack demonstrates a full-stack recruitment workflow using React, Node.js, Express and MongoDB. It brings public job discovery, candidate application submission, resume handling and application tracking together with administrator tools for job and application management. The project also demonstrates REST API integration, database persistence, JWT-based administrator authentication, file upload handling, testing and cloud deployment. The remaining improvement areas—especially persistent resume storage, stronger upload validation and broader test coverage—have been identified for future work.

19. References and Resources

Project repository

https://github.com/Nikhat-codes/CareerTrack

Live frontend

https://career-track-two.vercel.app

Backend health endpoint

https://careertrack-vvpy.onrender.com/api/health

Project documentation

README.md, API_Documentation.pdf, Database_Schema.pdf and Test_Cases.xlsx in the project repository.

Technology references

Official documentation for React, Vite, React Router, Axios, Node.js, Express, MongoDB, Mongoose, JSON Web Token, bcrypt and Multer.

SYSTEM ARCHITECTURE & DATA FLOW DIAGRAM



Public application requests follow the same route-to-controller-to-database path without administrator authentication. Protected administrator operations pass through JWT verification. Resume submissions additionally pass through upload middleware.

DATABASE RELATIONSHIP OVERVIEW

JOBS One job posting	**APPLICATIONS** Multiple submissions per job
Job record / publication state	Application details / status / notes / resume reference
ADMIN ACCOUNTS Authorized management access	Manages job and application workflows

Relationship interpretation: one job can have multiple application records; each application is associated with a job. Administrator accounts authenticate access to management functions. This is a conceptual view based on the documented modules, not a substitute for the exact field-level Mongoose schema.

API ENDPOINT SUMMARY

Method	Endpoint	Access	Purpose
GET	/api/jobs	Public	Published job list/search
GET	/api/jobs/:id	Public	Job details
POST	/api/applications	Public	Submit application + resume
POST	/api/applications/track	Public	Track by ID and email
POST	/api/auth/login	Public login	Administrator sign-in
GET	/api/dashboard/stats	Protected	Dashboard statistics
GET	/api/applications	Protected	Application list and filters
PATCH	/api/applications/:id/status	Protected	Update status and notes
GET	/api/applications/export/csv	Protected	Export application data
GET	/api/applications/:id/resume	Protected	Download resume
DELETE	/api/applications/:id	Protected	Delete application

Job-administration route names and exact request/response schemas should be checked against the current route files and Postman collection before treating this summary as a definitive API contract.

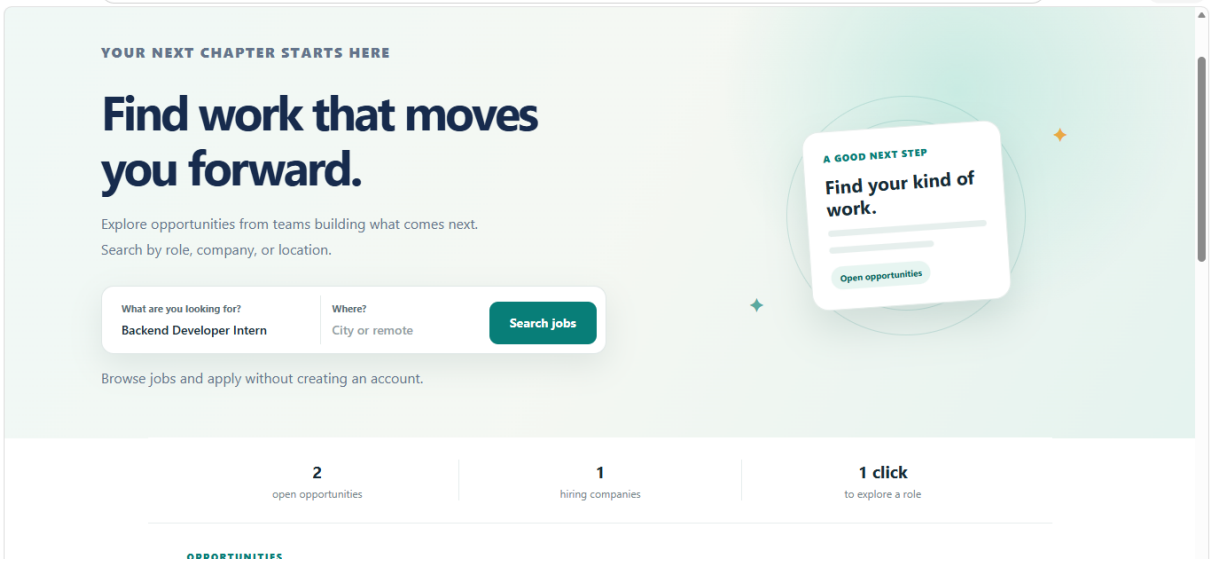
TESTING SUMMARY

Area	Recorded verification / current evidence
Frontend build	Production build was reported successful during development.
Backend health	Health endpoint returned a success response indicating that the API is running.
Application submission	Workflow listed as previously verified in the development handover.
Admin review	Application review workflow listed as previously verified.
Status persistence	Status update and persistence listed as previously verified.
Resume download	Workflow listed as previously verified; access should remain protected.
Application tracking	Workflow listed as previously verified using application ID and email.
CSV export	Workflow listed as previously verified.
Combined filters	Live combined job/date/status filter returned one matching application.

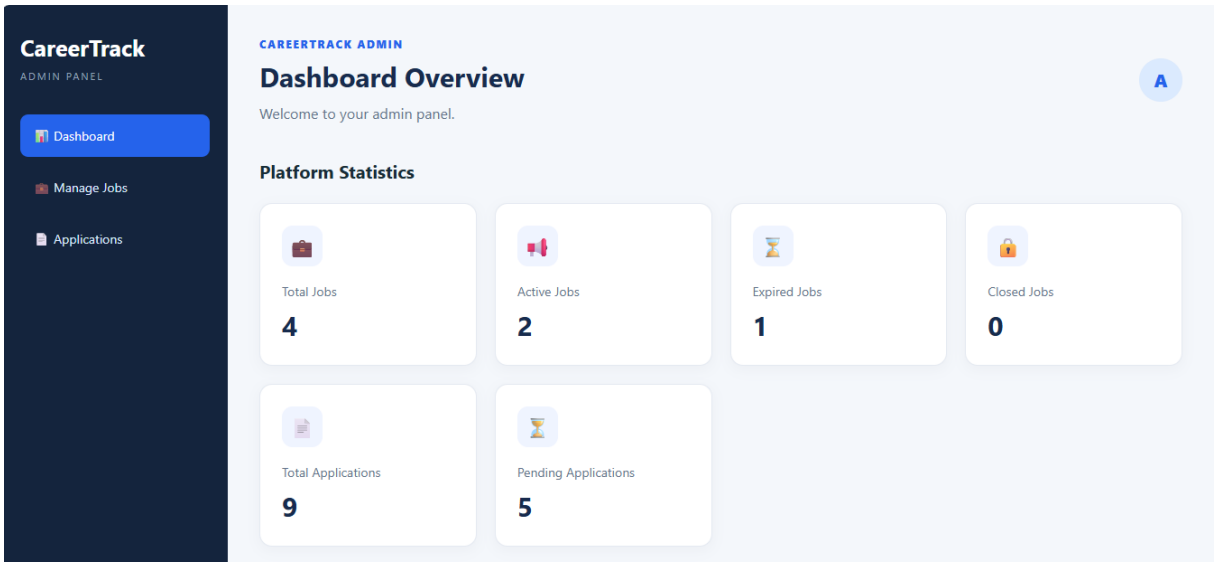
The report does not mark unexecuted negative-path, security or responsive test cases as passed. The complete Test_Cases.xlsx file should be used for the final test run and attached to the submission package.

SCREENSHOT EVIDENCE

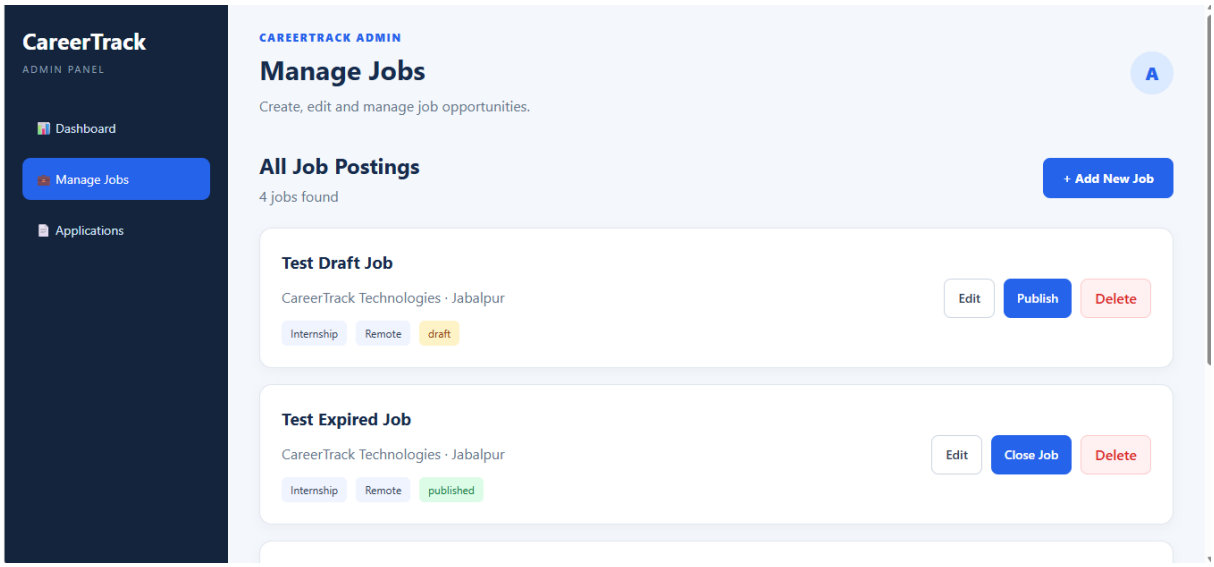
Screenshots below are included as visual evidence of the CareerTrack interface captured during development. They are presented as interface evidence, not as substitutes for executing the corresponding test cases.



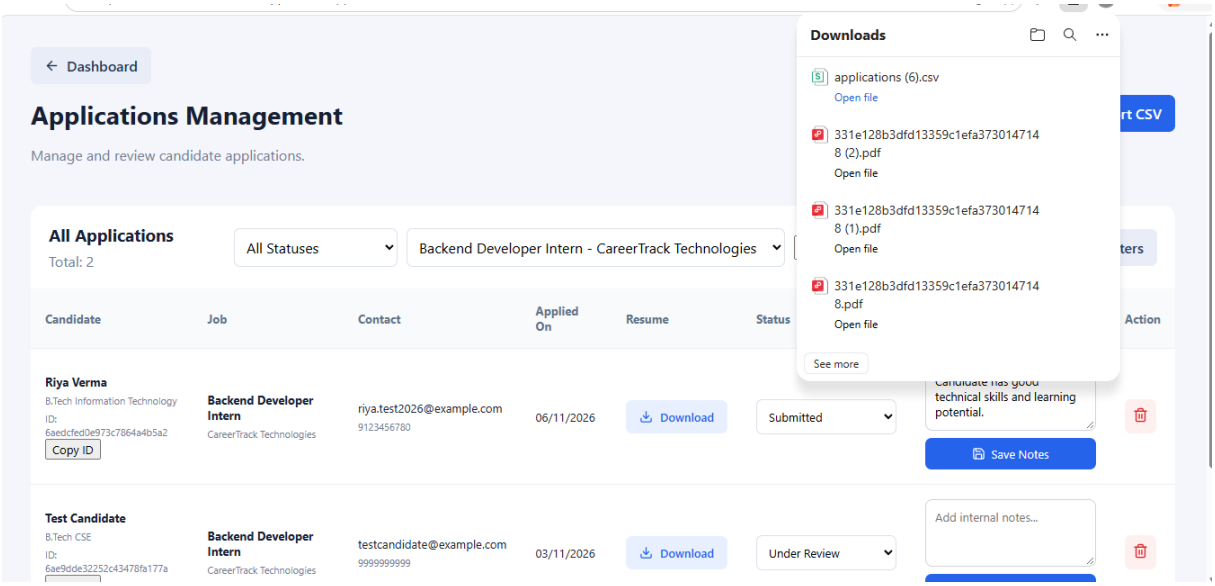
Public landing page / job discovery interface



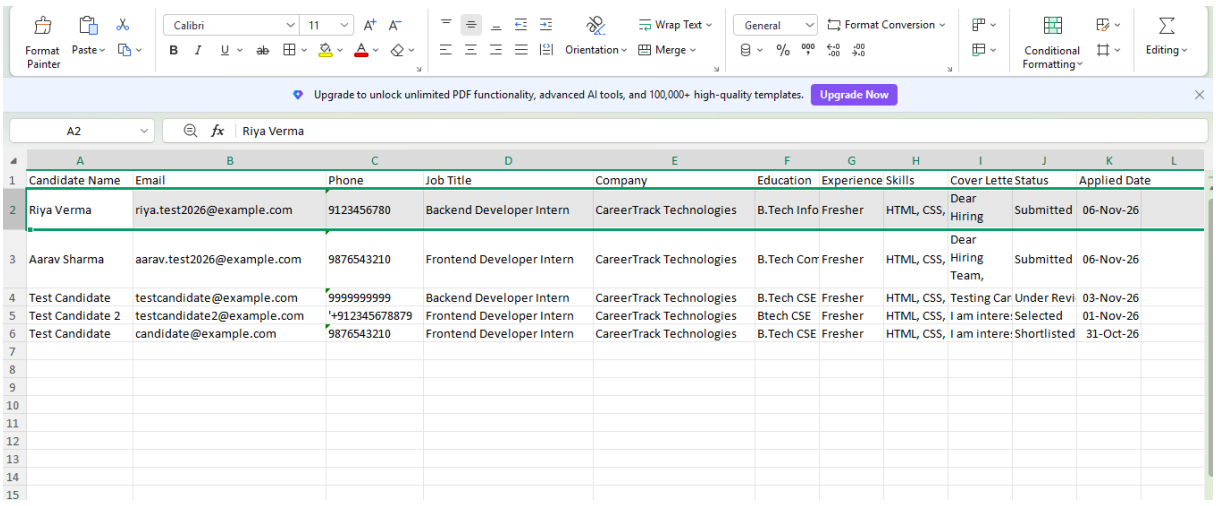
Administrator dashboard and summary cards



Job management interface



Application management interface



Application review and status-management view

DECLARATION OF IMPLEMENTATION SCOPE

This report describes CareerTrack based on the project documentation and development evidence available at the time of preparation. The report intentionally distinguishes implemented features from recommended improvements. Exact database field definitions, job-administration route mounts and some validation details should be verified against the final source code before evaluation.

END OF REPORT